SŠPU STŘEDNÍ ŠKOLA PRŮMYSLOVÁ A UMĚLECKÁ, OPAVA

ZÁVĚREČNÁ STUDIJNÍ PRÁCE

dokumentace

Haunted Mound

Ondřej Lampa

Obor:
18-20-M/01 INFORMAČNÍ TECHNOLOGIE
se zaměřením na počítačové sítě a programování

Třída: IT4

Školní rok: 2025/2026

Střední škola průmyslová a umělecká, Opava

2026

Poděkování
Rád bych poděkoval panu učiteli Lučnému za pomoc a chápavost.

ABSTRAKT

Tato práce se zabývá návrhem a implementací 2D akční roguelike hry Haunted Mound vytvořené v herním enginu Unity. Hra kombinuje prvky klasického platformeru s intenzivním boss-fightem proti Grim Reaperovi. Hlavní mechaniky zahrnují plynulý pohyb hráče, střílení, blízký útok mačetou, speciální útok havranem, systém tří srdíček životů přenášený mezi levely a komplexní UI prvky včetně health baru bosse. Klíčovou součástí projektu je integrace cloudové databáze Firebase Firestore, která umožňuje ukládat statistiky každého dokončeného runu (jméno hráče, počet zabitých nepřátel a čas dokončení). Do leaderboardu se zapisují pouze hráči, kteří úspěšně porazí bosse. Práce popisuje celý vývojový proces od návrhu herních mechanik přes implementaci AI nepřátel, animací, kolizí a uživatelského rozhraní až po cloudovou integraci a řešení technických problémů.

ABSTRACT

This thesis deals with the design and implementation of the 2D action roguelike game Haunted Mound created in the Unity game engine. The game combines elements of a classic platformer with an intense boss fight against the Grim Reaper. Core mechanics include smooth player movement, shooting, machete melee attacks, a special raven attack, a three-heart life system that persists between levels, and comprehensive UI elements including a boss health bar. A key part of the project is the integration of the Firebase Firestore cloud database, which enables storing statistics of each completed run (player name, number of enemies killed, and completion time). Only players who successfully defeat the boss are recorded in the leaderboard. The thesis describes the entire development process from game mechanics design, through implementation of enemy AI, animations, collisions and user interface, up to cloud integration and solving technical challenges.

Obsah

1 VÝVOJ 2D AKČNÍ ROGUELIKE HRY **5**
1.1 Základní principy vývoje 2D her v Unity 5
1.2 Koncepty roguelike her a boss fightů 5
1.3 Metodika vývoje – iterativní přístup a testování 6

2 Využité technologie **7**
2.1 Herní engine Unity a základní knihovny 7
2.2 Cloudová platforma Firebase Firestore 7
2.3 Další použité nástroje a assety 8

3 Způsoby řešení a použité postupy **9**
3.1 Návrch herních mechanik a architektury projektu 9
3.2 Implementace hráčského ovládání a systému životů 9
3.3 Vytvoření nepřátel, AI a boss fightu 9
3.4 Integrace firebase - ukládání dat a leaderboard 9
3.5 Vytvoření UI prvků a animací 10

4 Výsledky řešení, výstupy, uživatelský manuál **11**
4.1 Finální podoba hry a dosažené výsledky 11
4.2 Uživatelský manuál - ovládání a postup hry 11
4.3 Možnosti další rozšíření hry 12

5 Závěr **13**

6 Seznam použitých informačních zdrojů **14**

Úvod

Rozhodl jsem se vytvořit 2D akční roguelike hru Haunted Mound v herním enginu Unity, protože mě dlouhodobě zajímá tvorba her a chtěl jsem si vyzkoušet kompletní vývojový cyklus od nápadu až po finální produkt. Chtěl jsem vytvořit něco komplexnějšího – hru s více levely, boss-fightem, cloudovým ukládáním dat a funkčním menu. Hlavním cílem projektu bylo vytvořit hratelnou a vizuálně poutavou 2D akční hru s prvky roguelike. Hráč ovládá postavu v temném fantasy prostředí, bojuje proti vlnám nepřátel a na konci třetího levelu čelí finálnímu bossovi – Grim Reaperovi. Hra obsahuje klasické mechaniky jako plynulý pohyb, skok, střílení, blízký útok mačetou a speciální útok havranem. Důležitou součástí práce je také integrace cloudové databáze Firebase Firestore. Každý dokončený run se ukládá do databáze (jméno hráče, počet zabitých nepřátel a čas dokončení) a pouze úspěšní hráči, kteří porazí bosse, se objeví v žebříčku. Tímto způsobem jsem chtěl propojit lokální herní mechaniky s moderními cloudovými technologiemi. Na začátku projektu jsem se setkal s řadou technických problémů – od správného přetáčení spritů, přes přenášení životů mezi levely, až po stabilní komunikaci s Firebase. V průběhu vývoje jsem musel řešit kolize, AI nepřátele, animace, UI prvky a optimalizaci výkonu. V této dokumentaci podrobně popisuji jednotlivé kroky vývoje, použité technologie, vzniklé problémy a způsoby jejich řešení. V závěru pak hodnotím dosažené výsledky a nastiňuji možnosti dalšího rozšíření hry.

1 VÝVOJ 2D AKČNÍ ROGUELIKE HRY

1.1 Základní principy vývoje 2D her v Unity

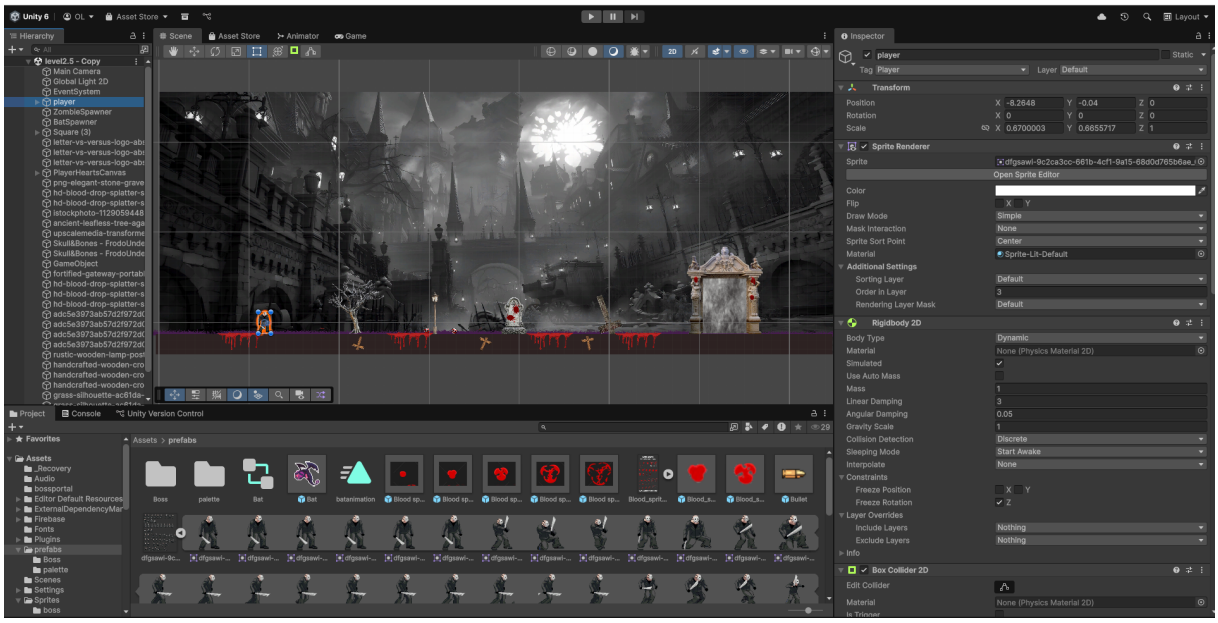
Unity je jeden z nejrozšířenějších herních enginů, který je velmi vhodný pro vývoj 2D her díky svému komponentovému systému. Každý objekt ve hře (GameObject) je složen z jednotlivých komponent, jako je Rigidbody2D pro fyziku, Collider2D pro detekci kolizí, Animator pro animace nebo Sprite Renderer pro zobrazení grafiky. Hlavní herní smyčka běží především ve dvou metodách – Update() pro běžnou logiku a FixedUpdate() pro fyzikální výpočty. Ovládání hráče je řešeno pomocí Input.GetAxisRaw() a Input.GetKeyDown(), což umožňuje plynulý pohyb a okamžitou reakci na stisk kláves. Pro správné chování postavy je důležitá také správná práce s transform.localScale při otáčení spritů a správné nastavení Rigidbody2D (gravity scale, freeze rotation).

1.2 Koncepty roguelike her a boss fightů

Roguelike hry jsou žánrem charakteristickým procedurální generací, vysokou obtížností a systémem instantní smrti. I když moje hra není plně procedurální, převzal jsem z roguelike her několik klíčových prvků – především systém runů, kde každý pokus začíná od začátku, a důraz na skill-based gameplay. Důležitou součástí hry je finální boss fight proti Grim Reaperovi. Boss fighty obvykle vyžadují specifickou strategii, kombinaci různých útoků (melee, střelba, speciální ability) a dobrou znalost chování bosse. V mé hře boss disponuje teleportací, blízkým útokem a různými fázemi boje, což od hráče vyžaduje jak dobrou reakci, tak taktiku.

1.3 Metodika vývoje – iterativní přístup a testování

Vývoj hry probíhal iterativním způsobem. Nejprve jsem vytvořil základní prototyp pohybu a ovládání hráče, poté přidal střílení a blízký útok, následně nepřátele a kolizní systém. Teprve poté jsem implementoval speciální mechaniky (havran, přenášení životů mezi levely) a nakonec cloudovou integraci Firebase. Každá nová funkce byla ihned testována v herním módu. Pro ladění jsem hojně využíval Debug.Log(), manuální testování kolizí, AI chování nepřátel a správnost ukládání dat do Firebase. Tento přístup mi umožnil rychle odhalovat a opravovat chyby a postupně zvyšovat kvalitu a stabilitu hry.



2 Využité technologie

2.1 Herní engine Unity a základní knihovny

Unity je jeden z nejpoužívanějších herních enginů pro vývoj 2D i 3D her. Využívá komponentový systém, kde je každý objekt (GameObject) složen z různých částí, jako jsou Transform, Rigidbody2D nebo Collider2D, které určují jeho chování.

Herní logika se vytváří pomocí skriptů v jazyce C#, které se připojují k jednotlivým objektům. Důležité jsou metody jako Update() pro běžný chod hry a FixedUpdate() pro fyziku.

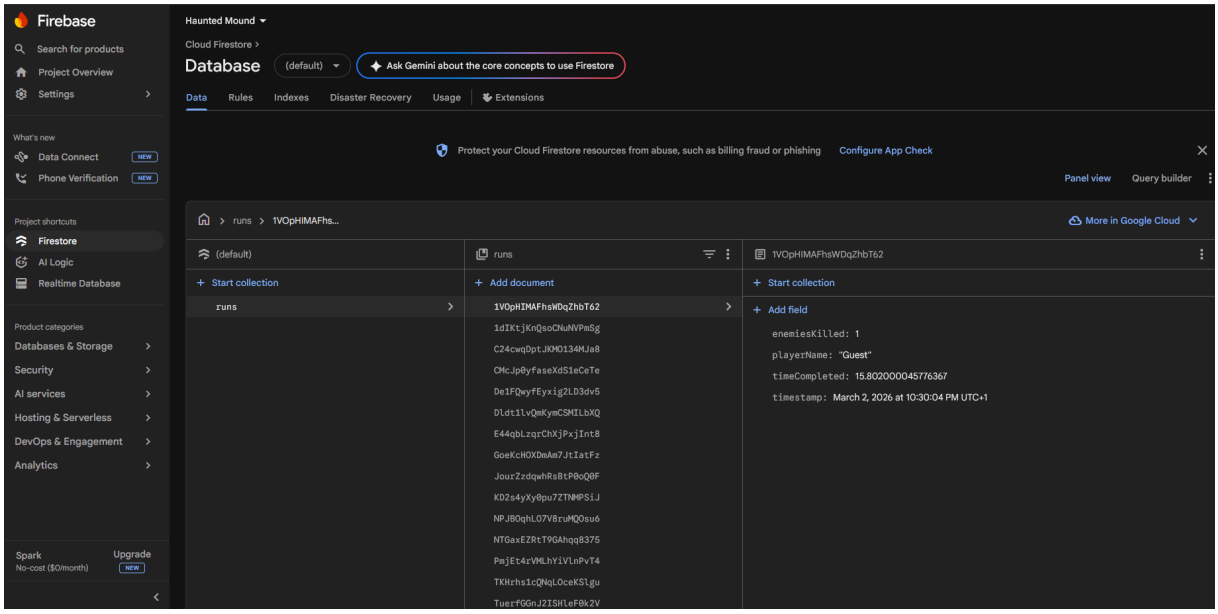
Unity obsahuje také základní knihovny pro animace (Animator), uživatelské rozhraní (Canvas) nebo zvuk (AudioSource). Díky tomu umožňuje rychlý a efektivní vývoj her.

2.2 Cloudová platforma Firebase Firestore

Pro ukládání statistik jednotlivých runů byla použita cloudová databáze Firebase Firestore. Jedná se o dokumentovou NoSQL databázi, která umožňuje rychlé zápisy a čtení dat, realtime synchronizaci a jednoduché dotazy. V projektu slouží Firestore k ukládání dat o každém dokončeném runu – jméno hráče, počet zabitých nepřátel, čas dokončení a příznak victory. Do leaderboardu se zapisují pouze hráči, kteří úspěšně porazili bosse. Integrace probíhala přes oficiální Firebase Unity SDK pomocí asynchronních metod (async / await) a AddAsync().

2.3 Další použité nástroje a assety

Kromě Unity a Firebase byly ve projektu využity i další nástroje a volně dostupné assety. Grafika byla vytvořena převážně z pixel art spriteů získaných z platforem OpenGameArt.org a itch.io. Pro tvorbu a úpravu animací byl používán vestavěný Animator v Unity. Verzování projektu probíhalo pomocí Git a GitHub. Pro psaní a ladění kódu byl použit editor Visual Studio. Zvukové efekty a hudba byly implementovány pomocí komponenty AudioSource. Testování probíhalo především v Unity Play módu a v samostatném buildu pro Windows.



3 Způsoby řešení a použité postupy

3.1 Návrch herních mechanik a architektury projektu

Na začátku vývoje jsem vytvořil základní herní design dokument, ve kterém jsem definoval hlavní mechaniky, ovládání a cíle hry. Architektura projektu je postavena na singletonu GameManager, který zajišťuje přenášení dat (jméno hráče, počet životů, statistiky runu) mezi jednotlivými scénami pomocí DontDestroyOnLoad(). Hráč je řízen skriptem PlayerController, nepřátelé mají vlastní řídicí skripty (EnemyController, FlyingEnemy, Stalker) a boss má samostatný skript Boss. Tento modulární přístup umožnil snadnější testování a rozšiřování jednotlivých systémů.

3.2 Implementace hráčského ovládání a systému životů

Pohyb hráče je realizován pomocí Rigidbody2D a Input.GetAxisRaw("Horizontal"). Skok je implementován přes AddForce() při stisku mezerníku za podmínky, že je hráč na zemi (isGrounded). Otočení postavy probíhá změnou transform.localScale podle směru pohybu nebo pozice myši. Systém životů je řešen třemi srdíčky reprezentovanými UI Image objekty. Při zásahu se snižuje currentHealth o 1 a aktualizuje se vizuální podoba srdíček. Životy se přenáší mezi levely díky GameManageru. Při dosažení nuly životů se spustí obrazovka „You Died“ a run se uloží do databáze.

3.3 Vytvoření nepřátel, AI a boss fightu

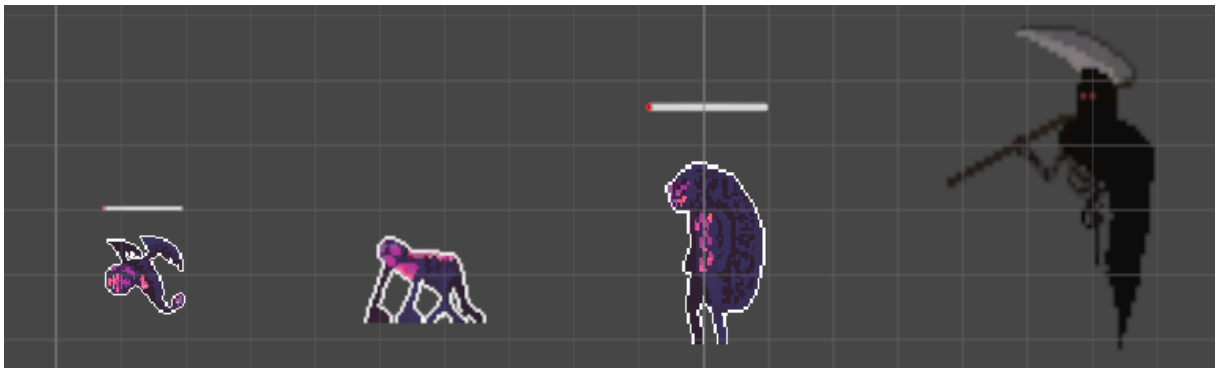
Nepřátelé jsou rozděleni do tří typů: klasický walker (zombie), létající netopýr a rychlý stalker. Každý typ má vlastní skript s jednoduchou AI – pronásledování hráče, detekce kolize a způsob útoku. Finální boss fight v třetím levelu je řízen skriptem Boss, který kombinuje pomalý patrol, náhodnou teleportaci za hráče a blízký útok kosou. Boss má vlastní health bar, animace poškození a animaci smrti. Boj vyžaduje od hráče kombinaci střelby, blízkého útoku a využívání havrana.

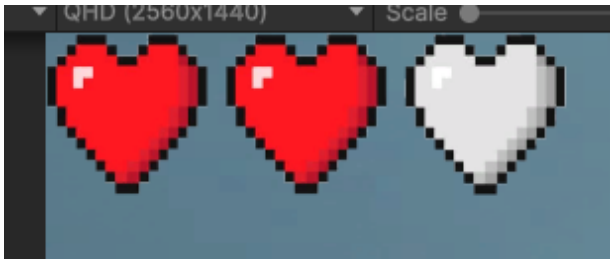
3.4 Integrace firebase - ukládání dat a leaderboard

Integrace Firebase Firestore probíhala asynchronně pomocí async Task. Po smrti hráče nebo zabití bosse se vytvoří záznam obsahující jméno hráče, počet zabitých nepřátel, čas dokončení a příznak victory. Do leaderboardu se ukládají pouze úspěšné runy (victory = true). Načítání dat probíhá v LeaderboardUI pomocí query s řazením podle počtu zabitých nepřátel nebo času. Pro správnou funkcionalitu byly v Firebase konzoli vytvořeny composite indexy.

3.5 Vytvoření UI prvků a animací

Veškeré uživatelské rozhraní je vytvořeno v Canvasu v režimu Screen Space – Overlay. Použity byly TextMeshPro pro texty, Image pro srdíčka a health bar, a Layout Groupy pro správné zarovnání. Animace postavy, nepřátel i bosse jsou řešeny pomocí Animator Controlleru s triggery a float parametry (např. „Speed“, „Health“).





4 Výsledky řešení, výstupy, uživatelský manuál

4.1 Finální podoba hry a dosažené výsledky

Výsledkem projektu je plně funkční 2D akční roguelike hra Haunted Mound se třemi levely a finálním boss-fightem. Hra obsahuje kompletní ovládání hráče (pohyb, skok, střílení, melee útok mačetou a speciální útok havranem), systém tří srdíček životů přenášený mezi levely, různé typy nepřátel (zombie, netopýr, stalker) a náročného bosse Grim Reapera s vlastní AI a animacemi. Díky integraci Firebase Firestore se statistiky každého runu ukládají do cloudu a hráči, kteří porazí bosse, se objevují v leaderboardu. Hra je stabilní, dobře optimalizovaná a poskytuje uspokojivý herní zážitek jak pro začátečníky, tak pro zkušenější hráče. Všechny hlavní mechaniky fungují tak, jak byly původně navrženy.

4.2 Uživatelský manuál - ovládání a postup hry

Hra se ovládá pomocí klávesnice a myši. Základní ovládání je následující:

Pohyb – klávesy A (vlevo) a D (vpravo) Skok – mezerník Střílení – levé tlačítko myši (směr podle pozice kurzoru) Blízký útok mačetou – pravé tlačítko myši Speciální útok havranem – klávesa Q (cooldown 3 sekundy) Menu – Escape (v případě potřeby)

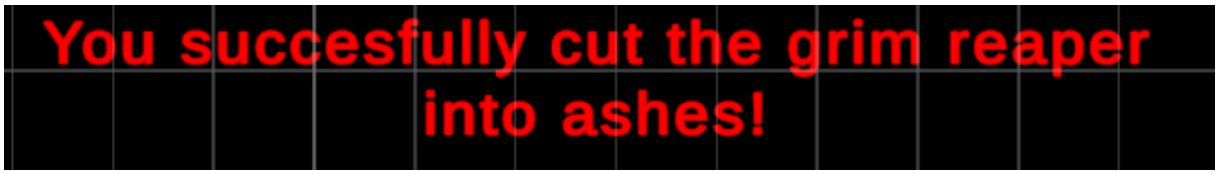
Hra začíná v hlavním menu, kde hráč může zadat své jméno nebo hrát jako Guest. Po spuštění hry prochází třemi levely s rostoucí obtížností. Cílem je přežít všechny nepřátele a porazit Grim Reapera v závěrečném boss fightu. Při ztrátě všech životů se zobrazí obrazovka „You Died“ a hráč má možnost zkusit run znovu nebo se vrátit do menu.

4.3 Možnosti další rozšíření hry

Přestože je hra v současné podobě plně hratelná, existuje řada možností, jak ji dále vylepšit. Mezi hlavní patří:

Přidání dalších levelů s novými prostředími a typy nepřátel Vytvoření originální hudby a zvukových efektů Implementace autentizace hráčů přes Firebase Auth pro bezpečnější leaderboard Přidání achievementů a dalších statistik Optimalizace pro webovou verzi nebo mobilní zařízení Vylepšení grafiky

Tyto rozšíření by mohla hru posunout směrem k plnohodnotnému indie titulu s online funkcemi.



5 Závěr

Cílem této závěrečné práce bylo vytvořit plně funkční 2D akční roguelike hru Haunted Mound v herním enginu Unity, včetně několika levelů, různých typů nepřátel a náročného boss fightu. Tento cíl se podařilo splnit. Hra obsahuje kompletní ovládání hráče (pohyb, skok, střílení, melee útok a speciální útok havranem), systém tří srdíček životů přenášený mezi levely, tři úrovně s rostoucí obtížností a finálního bosse Grim Reapera s vlastní AI a animacemi. Důležitou součástí projektu byla také integrace cloudové databáze Firebase Firestore, která umožňuje ukládat statistiky každého runu a zobrazovat leaderboard pouze úspěšným hráčům. Během vývoje se podařilo vyřešit řadu technických výzev, jako je správné přetáčení spriteů, přenášení dat mezi scénami, stabilní komunikace s Firebase a vyvážení obtížnosti boss fightu. Hra je stabilní, hratelná a poskytuje uspokojivý herní zážitek. Na druhou stranu se nepodařilo zrealizovat všechny původní plány. Hra měla mít pět levelů místo tří, plně vlastní grafiku a originální soundtrack. Z časových důvodů jsem musel použít volně dostupné assety a zaměřit se především na funkčnost a stabilitu. Přesto považuji projekt za úspěšný. Haunted Mound představuje komplexní praktickou ukázku vývoje 2D hry od návrhu až po cloudovou integraci. Do budoucna by bylo možné hru rozšířit o další levely, autentizaci hráčů, vlastní grafiku, lepší zvukový design nebo dokonce multiplayer prvky.

6 Seznam použitých informačních zdrojů

[1] Unity Technologies. Unity Documentation. Dostupné z: https://docs.unity3d.com

[2] Google. Firebase Documentation – Unity SDK a Cloud Firestore. Dostupné z: https://firebase.google.com/docs/unity

[3] Brackeys. Unity 2D Tutorials (archivní série). YouTube. Dostupné z: https://www.youtube.com/c

[4] Blackthornprod. 2D Pixel Art Tutorials and Game Design Tips. YouTube. Dostupné z: https://www.youtube.com/c/Blackthornprod

[5] OpenGameArt.org. Free Pixel Art Sprites and Assets. Dostupné z: https://opengameart.org

[6] Itch.io. Free Game Assets (SFX, Music and Sprites). Dostupné z: https://itch.io/game-assets/free

[7] Microsoft. C# Language Reference. Dostupné z: https://learn.microsoft.com/en-us/dotnet/csharp/

[8] TextMeshPro Documentation. Unity Manual. Dostupné z: https://docs.unity3d.com/Packages/co

[9] Stack Overflow. Komunita pro řešení problémů s Unity a C# (různé příspěvky, 2025–2026).

[10] Reddit – r/Unity2D a r/Firebase. Diskuze a tipy k vývoji 2D her a cloudové integraci (2025–2026).

[11] xAI – Grok. Asistence při vývoji, ladění kódu, řešení chyb Firebase a strukturování dokumentace (2025–2026).